DATABASE RELATIONSHIP ANALYSIS USING JOINS FOR E-COMMERCE DATABASE

INTRODUCTION

SQL JOIN operations are used to retrieve related information from multiple tables in a relational database. In an E-Commerce system, JOIN queries help combine customer details, product information, order records, and payment details. This experiment demonstrates the use of INNER JOIN, LEFT JOIN, and RIGHT JOIN to retrieve order information, display customer purchase history, and generate useful business reports.

1. Combine Customer, Product, Order, and Payment Tables

1.1 Combine Customer and Order Details

The JOIN operation combines the CUSTOMER and ORDERS tables using CUSTOMER_ID. It displays customer information along with their order details.

1.2 Combine Order and Payment Details

The JOIN operation combines the ORDERS and PAYMENT tables using ORDER_ID. It displays the order amount, payment method, payment status, and payment amount.

1.3 Combine Customer, Product, Order, and Payment Details

Multiple JOIN operations are used to combine related records from different tables. This helps retrieve customer names, product details, order information, quantities, and payment details in a single result.

```
SQL> SELECT C.ID AS CUSTOMER_ID,
  2         C.NAME AS CUSTOMER_NAME,
  3         O.ORDER_ID,
  4         O.ORDER_DATE,
  5         PR.PRODUCT_ID,
  6         PR.PRODUCT_NAME,
  7         PR.BRAND,
  8         OI.QUANTITY,
  9         PR.PRICE AS PRODUCT_PRICE,
 10         O.TOTAL_AMOUNT AS ORDER_AMOUNT,
 11         P.PAYMENT_METHOD,
 12         P.PAYMENT_STATUS,
 13         P.AMOUNT AS PAYMENT_AMOUNT
 14  FROM CUSTOMER C
 15  JOIN ORDERS O
 16      ON C.ID = O.CUSTOMER_ID
 17  JOIN ORDER_ITEMS OI
 18      ON O.ORDER_ID = OI.ORDER_ID
 19  JOIN PRODUCT PR
 20      ON OI.PRODUCT_ID = PR.PRODUCT_ID
 21  LEFT JOIN PAYMENT P
 22      ON O.ORDER_ID = P.ORDER_ID
 23  ORDER BY O.ORDER_ID;
```

2. Implement INNER JOIN, LEFT JOIN, and RIGHT JOIN

2.1 INNER JOIN

The INNER JOIN operation returns only the rows that have matching values in both tables. It displays customers who have placed orders and the corresponding order details.

```
SQL> SELECT C.ID,
  2         C.NAME,
  3         C.ADDRESS,
  4         O.ORDER_ID,
  5         O.ORDER_DATE,
  6         O.TOTAL_AMOUNT
  7  FROM CUSTOMER C
  8  INNER JOIN ORDERS O
  9  ON C.ID = O.CUSTOMER_ID;
```

2.2 LEFT JOIN

The LEFT JOIN operation returns all rows from the left table and matching rows from the right table. If a customer has not placed an order, the order-related columns display NULL values. This helps identify customers who have not made any purchases.

```
SQL> SELECT C.ID,
  2         C.NAME,
  3         O.ORDER_ID,
  4         O.ORDER_DATE,
  5         O.TOTAL_AMOUNT
  6  FROM CUSTOMER C
  7  LEFT JOIN ORDERS O
  8  ON C.ID = O.CUSTOMER_ID;
```

2.3 RIGHT JOIN

The RIGHT JOIN operation returns all rows from the right table and matching rows from the left table. It ensures that every order is displayed, even if a matching customer record is unavailable.

```
SQL> SELECT C.NAME,
  2         O.ORDER_ID,
  3         O.CUSTOMER_ID,
  4         O.ORDER_DATE,
  5         O.TOTAL_AMOUNT
  6  FROM CUSTOMER C
  7  RIGHT JOIN ORDERS O
  8  ON C.ID = O.CUSTOMER_ID;
```

3. Retrieve Complete Order Details

A multi-table JOIN query is used to retrieve complete order information, including customer ID, customer name, order ID, order date, product ID, product name, brand, quantity, product price, total order amount, payment method, and payment status.

This query provides a consolidated view of each order and its associated details.

4. Display Customer Purchase History

The customer purchase history query retrieves the orders placed by each customer. It displays customer names, order IDs, order dates, and total order amounts. The results can be sorted by customer ID or order date to understand previous purchases and track customer transactions.

```
SQL> SELECT C.NAME,
  2         C.ADDRESS,
  3         O.ORDER_ID,
  4         O.ORDER_DATE,
  5         O.TOTAL_AMOUNT,
  6         P.PAYMENT_METHOD,
  7         P.PAYMENT_STATUS,
  8         P.AMOUNT
  9  FROM CUSTOMER C
 10  INNER JOIN ORDERS O
 11  ON C.ID = O.CUSTOMER_ID
 12  LEFT JOIN PAYMENT P
 13  ON O.ORDER_ID = P.ORDER_ID
 14  ORDER BY O.ORDER_ID;
```

5. Generate Multi-Table Reports

5.1 Customer Order Summary

The COUNT and SUM aggregate functions are used with JOIN operations to calculate the total number of orders and total purchase amount for each customer.

```
SQL> SELECT C.ID,
  2         C.NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS,
  4         NVL(SUM(O.TOTAL_AMOUNT), 0) AS TOTAL_PURCHASE
  5  FROM CUSTOMER C
  6  LEFT JOIN ORDERS O
  7  ON C.ID = O.CUSTOMER_ID
  8  GROUP BY C.ID, C.NAME
  9  ORDER BY C.ID;
```

5.2 Payment Status Report

The CUSTOMER, ORDERS, and PAYMENT tables are combined to display payment methods, payment amounts, and payment statuses for customer orders. This report helps monitor successful and pending payments.

```
SQL> SELECT C.ID,
  2         C.NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS,
  4         NVL(SUM(O.TOTAL_AMOUNT), 0) AS TOTAL_PURCHASE
  5  FROM CUSTOMER C
  6  LEFT JOIN ORDERS O
  7  ON C.ID = O.CUSTOMER_ID
  8  GROUP BY C.ID, C.NAME
  9  ORDER BY C.ID;
```

5.3 Product and Order Report

The PRODUCT and order-related tables are combined to display product names, brands, quantities ordered, product prices, and order amounts. This report helps understand which products are included in customer orders.

Result

SQL JOIN operations were implemented successfully to combine customer, product, order, and payment information. INNER JOIN, LEFT JOIN, and RIGHT JOIN were used to retrieve related records, display customer purchase history, obtain complete order details, and generate multi-table business reports.